



Security Audit

USD8 (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	29
Conclusion	30
Our Methodology	31
Disclaimers	33
About Hashlock	34

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The USD8 partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

USD8 (USD8) is an EVM stablecoin system on HyperEVM composed of a role-gated ERC-20 token, an upgradeable subscription and redemption vault, an ERC-4626 staking vault, a cooldown Silo, a two-party reward distributor, and a lock-position system that allocates additional yield. Users can deposit whitelisted stablecoins through USD8MintVault and receive USD8, with deposited stablecoins forwarded to the treasury. Redemptions escrow USD8 in a request queue and later pay a configured stablecoin from reserves. USD8 can also be staked in StakedUSD8V2 to receive sUSD8, with reward injections vesting linearly over eight hours. sUSD8 can be locked in SUSD8Locker to receive a multiplier-based share of boost rewards. These lock positions are transferable ERC-721 tokens. Transferring a position moves the full entitlement to the new owner, and the only transfer restriction is that addresses frozen at the sUSD8 level cannot send or receive a position.

Project Name: USD8 (USD8)

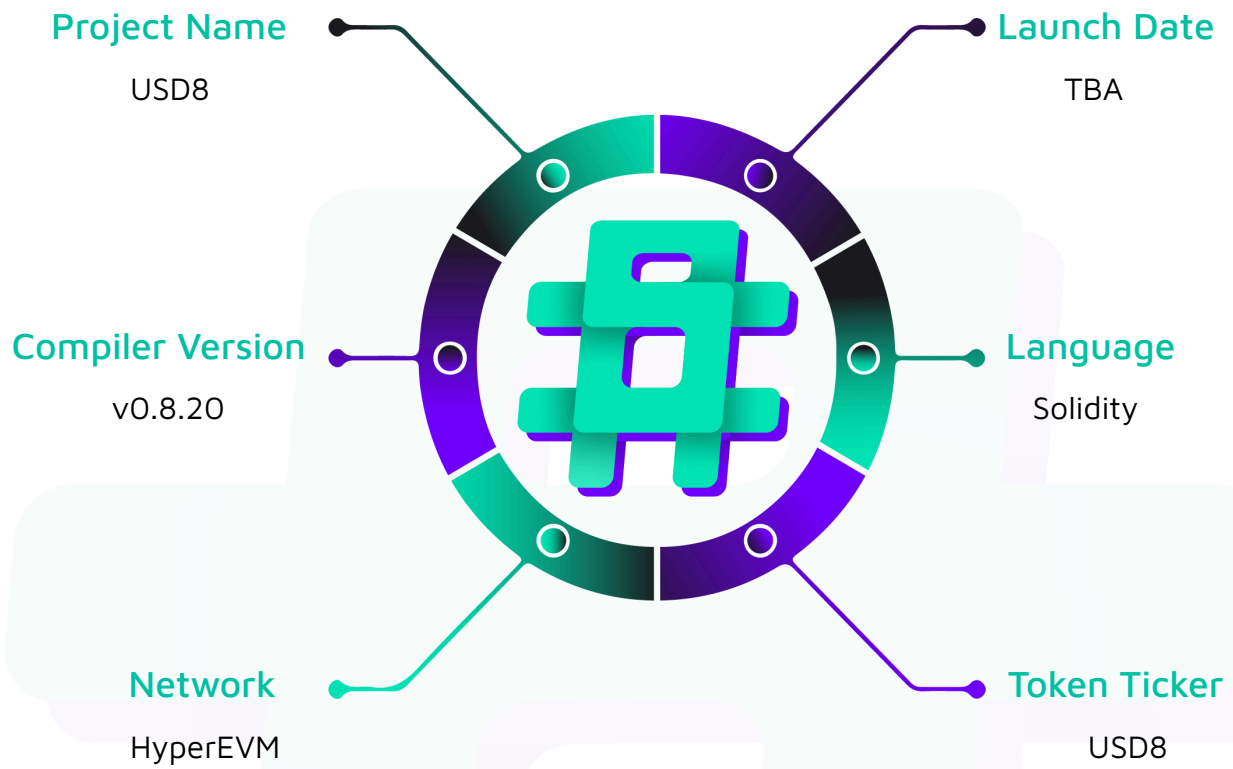
Project Type: DeFi / stablecoin, mint-redemption vault, ERC-4626 staking and reward infrastructure

Compiler Version: 0.8.20

Website: <https://usd8.ai/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the USD8, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	USD8 Smart Contracts
Network	HyperEVM
Language	Solidity
Audit Date	August, 2026
Contract 1	RewardsDistributor.sol
Contract 2	SUSD8Locker.sol
Contract 3	SingleAdminAccessControl.sol
Contract 4	StakedUSD8.sol
Contract 5	StakedUSD8V2.sol
Contract 6	USD8.sol
Contract 7	USD8MintVault.sol
Contract 8	USD8Silo.sol
Contract 9	AggregatorV3PriceAdapter.sol
Contract 10	ConstantPriceAdapter.sol
Contract 11	AggregatorV3Interface.sol
Contract 12	GuardianCanceller.sol
Contract 13	SUSD8LockerDescriptor.sol
Audited GitHub Commit Hash	7df6d72d0b72f93063214375cbde5549923b40a1
Fix Review GitHub Commit Hash	6eb34ca7483d43ef26c68d48312861aa0f9e1225

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

5 Medium severity vulnerabilities

7 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
RewardsDistributor.sol - Creates time-limited USD8 reward proposals through a dedicated proposer role. - Requires approval by a different approver and permits either operating role to veto pending proposals. - Checks each proposal against the configured reward-size circuit breaker.	Contract achieves this functionality.
SUSD8Locker.sol - Locks sUSD8 in ERC-721 positions with configurable tenors and yield multipliers. - Tracks active extra weight, expiry buckets, and a daily additive boost index.	Contract achieves this functionality.
SingleAdminAccessControl.sol - Maintains one current DEFAULT_ADMIN_ROLE holder. - Implements a two-step administrator transfer through transferAdmin() and acceptAdmin().	Contract achieves this functionality.
StakedUSD8.sol - Implements the ERC-4626 sUSD8 staking vault and ERC-20 permit-enabled share token. - Accepts USD8 deposits and issues sUSD8 shares.	Contract achieves this functionality.
StakedUSD8V2.sol - Adds a configurable cooldown-based withdrawal model to StakedUSD8. - Disables standard ERC-4626 withdraw() and	Contract achieves this functionality.

redeem() while cooldown mode is active.	
USD8.sol - Implements the 18-decimal USD8 ERC-20 token with EIP-2612 permit support. - Allows MINTER_ROLE holders to mint USD8 and holders or approved spenders to burn USD8.	Contract achieves this functionality.
USD8MintVault.sol - Accepts whitelisted stablecoin deposits and mints oracle-valued USD8 to a selected receiver. - Forwards subscription assets directly to the configured treasury.	Contract achieves this functionality.
USD8Silo.sol - Custodies USD8 produced by sUSD8 cooldown withdrawals. - Allows only the configured staking vault to release stored USD.	Contract achieves this functionality.
AggregatorV3PriceAdapter.sol - Reads a configured Chainlink-compatible price feed through an immutable feed address. - Pins and validates the feed decimal count during deployment.	Contract achieves this functionality.
ConstantPriceAdapter.sol - Provides a fixed 1e18 USD price for testnet or explicit emergency configuration. - Returns the current block timestamp so the reported fixed price	Contract achieves this functionality.
AggregatorV3Interface.sol - Defines Chainlink-compatible feed metadata functions.	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Defines historical and latest round-data retrieval methods used by oracle adapters. | |
|---|--|



Code Quality

This audit scope involves the smart contracts of the USD8, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the USD8 smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] `src/RewardsDistributor.sol#_checkDailyLimit` - The daily reward cap is enforced per proposal instead of cumulatively

Description

`maxDailyRewardBps` is presented as a daily circuit breaker, but the contract applies it independently to each proposal rather than to the total rewards executed during the day.

Vulnerability Details

Both `propose()` and `approve()` call `_checkDailyLimit(amount)`, which only compares the current proposal with a percentage of `sUSD8.totalAssets()`. The contract stores no daily or rolling-window usage and reserves no capacity for pending proposals.

Consequently, several proposals can each use the full advertised daily allowance. The eight-hour vesting interlock limits their frequency, but still permits multiple full-cap executions within 24 hours. For example, a 0.5% limit can permit approximately three 0.5% injections rather than limiting aggregate issuance to 0.5%.

Impact

Compromise or collusion of the proposer and approver can mint materially more USD8 than the circuit breaker is intended to allow, increasing dilution and reserve-accounting risk while all individual checks continue to pass.

Recommendation

Track aggregate reward execution over a defined rolling 24-hour window or calendar day and enforce the remaining capacity in `approve()`. Pending proposals should either reserve capacity or be revalidated against the unconsumed aggregate allowance at execution.

Status

Resolved



[M-02] `src/SingleAdminAccessControl.sol#revokeRole` - The guardian can indefinitely cancel its own removal

Description

The protocol routes core role revocations through the Timelock while the same brake-only guardian is configured with unrestricted cancellation authority over that Timelock.

Vulnerability Details

`SingleAdminAccessControl.revokeRole()` can only be called by the current `DEFAULT_ADMIN_ROLE`. Under the protocol's governance configuration, that admin is the Timelock, so removing the guardian's protocol roles requires a scheduled Timelock call. Revoking the guardian's Timelock cancellation authority must likewise pass through the same Timelock.

Because the guardian can cancel arbitrary pending operations, it can observe and cancel the operation that removes its own permissions, then repeat this for every replacement attempt. The same authority can cancel unpauses, upgrades, oracle replacements, role rotations, and other recovery operations.

Impact

A compromised hot guardian becomes an effectively permanent governance-veto key and can keep the protocol paused or prevent recovery indefinitely, despite being documented as a limited brake-only role.

Recommendation

Use a target- and selector-scoped cancellation module, an expiring emergency role, a threshold guardian, or an independent role-removal path that the guardian cannot cancel. The guardian should not have unrestricted veto power over operations that revoke its own permissions.

Status

Resolved

[M-03] src/USD8MintVault.sol#claimRedeem - Blacklisted redemption requests cannot be claimed, cancelled, or confiscated

Description

A redemption request moves the user's USD8 into the shared MintVault, but the compliance mechanism continues to operate only on the user's direct token balance.

Vulnerability Details

Currently, If a requester is blacklisted after creating a request, `claimRedeem()` reverts because it explicitly checks `isBlacklisted(r.user)`. `cancelRedeem()` also reverts because returning USD8 to the blacklisted user is rejected by the token hook.

At the same time, `USD8.redistributeLockedAmount()` cannot seize the request because the USD8 is held by the MintVault and the user's direct balance is zero. There is no request-specific administrative cancellation or confiscation path. A user who anticipates a freeze can also move their balance into the queue before the blacklist transaction is executed.

Impact

The request and its escrowed USD8 can remain unresolved for as long as the blacklist remains active, while governance cannot selectively execute a court-ordered confiscation without affecting unrelated requests held by the same vault.

Recommendation

Treat the request owner as the beneficial owner of the escrowed USD8. Add a request-specific compliance function that can close a frozen request and redirect or burn its escrowed USD8 while preserving unrelated requests and accounting.

Status

Resolved

[M-04] src/StakedUSD8.sol#maxDeposit/maxMint - ERC-4626 deposit limits overstate executable capacity

Description

StakedUSD8 inherits the default ERC-4626 `maxDeposit()` and `maxMint()` implementations even though it adds global and receiver-specific restrictions that can make deposits impossible.

Vulnerability Details

The inherited functions report effectively unlimited capacity. However, `_deposit()` reverts whenever `depositsPaused` is set, and the share-transfer hook rejects minting shares to a fully restricted receiver.

Accordingly, `maxDeposit(receiver)` and `maxMint(receiver)` remain positive when every nonzero deposit or mint for that receiver would revert. ERC-4626 requires these limit functions not to overstate the amount accepted by the corresponding operation and to return zero when deposits are disabled.

Impact

ERC-4626 routers, aggregators, and automated strategies can select an operation based on the advertised maximum and then revert during execution. In batched integrations, one impossible vault action can block the entire allocation or rebalancing transaction.

Recommendation

Override `maxDeposit()` and `maxMint()` to return zero while deposits are paused and for restricted or invalid receivers. Return the actual configured capacity in every other state.

Status

Resolved

[M-05] src/StakedUSD8V2.sol#maxWithdraw/maxRedeem - ERC-4626 exit limits remain positive while standard exits are disabled

Description

The default seven-day cooldown disables the standard ERC-4626 `withdraw()` and `redeem()` functions, but their corresponding maximum views continue to report positive amounts.

Vulnerability Details

`withdraw()` and `redeem()` revert whenever `cooldownDuration != 0` through `ensureCooldownOff`. Nevertheless, the inherited `maxWithdraw(owner)` and `maxRedeem(owner)` continue to return the value of the owner's shares. They also remain positive for a restricted owner even though `_withdraw()` rejects the operation.

This contradicts the ERC-4626 requirement that a temporarily disabled exit report a maximum of zero.

Impact

Integrations can treat assets as synchronously withdrawable when only the custom cooldown flow is available, causing failed withdrawals, blocked batches, and broken liquidity or rebalancing assumptions.

Recommendation

Override `maxWithdraw()` and `maxRedeem()` to return zero while the standard exit path is disabled or the owner is restricted. Because `cooldownAssets()` and `cooldownShares()` currently call these standard views, introduce separate internal or public cooldown-limit helpers so the custom exit path remains usable.

Status

Resolved

Low

[L-01] `src/StakedUSD8V2.sol#unstake` - A blacklisted cooldown owner can redirect queued USD8 to a clean receiver

Description

A user who enters cooldown before being blacklisted can later release the queued USD8 to an arbitrary unrestricted receiver.

Vulnerability Details

`cooldownAssets()` and `cooldownShares()` burn the user's shares immediately and transfer the corresponding USD8 to the shared Silo. A later blacklist therefore finds no remaining sUSD8 balance to redistribute.

After maturity, `unstake(receiver)` checks only the cooldown timestamp. It does not check whether `msg.sender` is restricted and does not bind the payout to the cooldown owner. The eventual USD8 transfer is made from the clean Silo to the selected clean receiver, so the token's blacklist hook never sees the frozen beneficial owner.

Impact

The restricted user can recover the full economic value of a pre-existing cooldown through another address. This is Low if restrictions are deliberately address-only, but should be considered Medium if blacklisting is intended to immobilize all protocol claims beneficially owned by that principal.

Recommendation

Check the cooldown owner's restriction state in `unstake()` and add an administrative seizure or redirection path for frozen cooldown balances. The receiver should be fixed or validated against the beneficial owner at cooldown creation and claim time.

Status

Resolved

[L-02] `src/SUSD8Locker.sol#_open` - Calendar-day expiry can grant a position one extra boost allocation

Description

Locker maturity is checked using an exact timestamp, but reward eligibility expires only through a calendar-day bucket keyed to `maturityDay + 1`.

Vulnerability Details

A position that has already passed its exact maturity remains included in `totalExtraWeight` until it is claimed or the following day's expiry bucket is processed. If the daily reward injection occurs later on the maturity day, the matured position still receives that allocation.

A position opened before the lock-day injection can therefore receive the lock-day allocation plus every allocation through the maturity day, resulting in up to one more daily boost allocation than the stated tenor implies.

Impact

A rational user can delay claiming until after the maturity-day injection and receive excess boost shares at the expense of other active lockers. The issue is Low in ordinary conditions, but may warrant Medium if daily boost distributions are large and the extra allocation is systematic.

Recommendation

Define one unambiguous reward-eligibility boundary. Either process expiries by exact timestamp before each injection or enforce a fixed daily cutoff and calculate eligibility so every tenor receives exactly the intended number of distributions.

Status

Resolved

[L-03] `src/StakedUSD8.sol#_checkMinShares` - Non-atomic manual seeding lets a tiny donation block `sUSD8` initialization

Description

`StakedUSD8` relies on an external initial seed but does not make initialization atomic or protect an empty vault from assets arriving before the first shares are minted.

Vulnerability Details

While `sUSD8.totalSupply() == 0`, anyone able to obtain USD8 can transfer or mint USD8 directly to the vault without receiving shares. Under the inherited ERC-4626 virtual accounting, the first actual deposit then mints approximately `depositAmount / (donation + 1)` share units.

`StakedUSD8` requires every nonzero supply to be at least `1e18` shares. A sufficiently small direct donation can therefore make the intended initialization deposit mint fewer than `MIN_SHARES` and revert. The donated USD8 cannot be recovered through `rescueTokens()` because the vault asset is explicitly excluded. Any non-atomic external seeding procedure remains exposed to this state.

Impact

An attacker can grief the initialization sequence and force an exceptional recovery or redeployment. The attack does not directly generate profit and depends on the non-atomic manual deployment procedure, so Low severity is appropriate.

Recommendation

Initialize `sUSD8` atomically before any untrusted USD8 transfer or mint path can target it, and place the resulting anchor shares in an on-chain mechanism that prevents their withdrawal. A narrowly scoped empty-vault recovery mechanism may also be appropriate.

Status

Resolved

[L-04] `src/USD8MintVault.sol#cancelRedeem` - Cancelled requests permanently consume the day's redemption capacity

Description

The daily redemption counter is increased when a request is created but is not released when that request is cancelled.

Vulnerability Details

`requestRedeem()` calls `_checkLimits()`, which immediately adds `usd8Amount` to `dailyRedeemed[day]`. `cancelRedeem()` returns the complete escrowed USD8 but leaves the counter unchanged.

An attacker can consume the remaining daily capacity and cancel in the same transaction, recovering all principal while every later user is prevented from entering the redemption queue until the next day. The issue exists whenever `maxDailyRedeem` is nonzero; already queued claims remain executable.

Impact

The redemption-entry path can be denied repeatedly for the rest of each calendar day at only gas cost once the attacker has temporary access to enough USD8.

Recommendation

Store the quota day in each request and release same-day capacity when a request is cancelled. If the cap is intentionally based on gross request volume, impose a meaningful non-refundable cost and document that cancellation does not restore capacity.

Status

Resolved

[L-05] `src/StakedUSD8.sol#totalAssets` - Final exits can leave zero shares while rewards continue vesting

Description

The vault permits total supply to return to zero even while previously transferred rewards remain unvested inside the contract.

Vulnerability Details

`totalAssets()` excludes `getUnvestedAmount()`. The final shareholder can therefore cooldown or redeem all shares against only the vested assets, and `_checkMinShares()` explicitly permits `totalSupply() == 0`. The unvested USD8 remains in the vault and later becomes part of `totalAssets()` without any shares outstanding.

A subsequent depositor then receives very few shares under ERC-4626 virtual accounting and can fail the `MIN_SHARES` check, making an ordinary restart economically impractical. The deployment comment describes a permanent seed, but the script does not create it and the contracts do not make seed shares non-withdrawable.

Impact

If every share, including the operational seed, is removed during vesting, `sUSD8` can become unusable for normal deposits until an exceptional recovery is performed. The required system state and intended seed assumption make this Low severity.

Recommendation

Prevent the final exit while rewards remain unvested, enforce a genuinely non-withdrawable anchor, or define a safe zero-supply path that redirects remaining rewards and resets vesting before deposits resume.

Status

Resolved

[L-06] **src/StakedUSD8.sol#redistributeLockedAmount** - Zero-address redistribution can bypass the minimum-share invariant

Description

The compliance path can burn a restricted user's complete share balance without applying the minimum-supply check used by ordinary deposits and exits.

Vulnerability Details

When `to == address(0)`, `redistributeLockedAmount()` calls `_burn(from, amountToDistribute)` and converts the corresponding value into a vesting reward. It does not call `_checkMinShares()` afterward.

If the remaining supply is nonzero but below `MIN_SHARES`, the vault is left in a state that normal user flows are designed not to create. Subsequent small deposits and partial exits can revert until a sufficiently large deposit or a complete remaining exit repairs the supply.

Impact

A legitimate compliance action can temporarily impair vault usability and force remaining holders or governance to coordinate a recovery. Because the state can be repaired and requires an administrative confiscation, Low severity is appropriate.

Recommendation

Apply the same post-supply invariant to the zero-address redistribution branch. If burning would leave a subminimum supply, reject the operation or use a dedicated path that safely handles the remaining shares and underlying assets.

Status

Resolved

[L-07] src/USD8.sol#redistributeLockedAmount - Frozen users can burn balances during the timelock delay and defeat nonzero confiscation

Description

The blacklist blocks transfers from a restricted user but deliberately leaves burn-to-zero available. Confiscation to a nonzero recipient is executed only after the 48-hour Timelock delay.

Vulnerability Details

USD8 inherits the public `burn()` function from `ERC20Burnable`, and `_beforeTokenTransfer()` permits a restricted address to transfer to `address(0)`. During the public delay before `redistributeLockedAmount(from, recipient)` executes, the frozen user can burn some or all of the balance.

The confiscation function reads `balanceOf(from)` only at execution time. It therefore mints the recipient only the reduced remaining balance, potentially zero. This report applies to the nonzero-recipient confiscation branch; burning is already the intended outcome when `to == address(0)`.

Impact

A restricted user can unilaterally destroy funds that were intended to be transferred to a treasury, claimant, or other court-ordered recipient, defeating the protocol's confiscation outcome even though the user cannot personally retain the value.

Recommendation

Disallow voluntary burns by blacklisted or confiscation-pending accounts. Preferably escrow or snapshot the confiscatable amount when the freeze is imposed so the user cannot reduce it before the delayed execution.

Status

Resolved

QA

[Q-01] `src/SUSD8Locker.sol#accrue` - Day-by-day backfill can make reward approval impractical after long inactivity

Description

Locker checkpointing performs work proportional to every calendar day elapsed since the previous successful accrual.

Vulnerability Details

`currentExtraWeight()` loops over all skipped days, and `accrue()` repeats the loop while also writing `indexAtDay[d]` for each missing day. There is no maximum batch size or partial checkpoint function.

After a sufficiently long period without successful rewards, `RewardsDistributor.approve()` can exceed the available gas because it first calls `currentExtraWeight()` and then `accrue()`. The contract cannot process only part of the backlog, so the same full loop is required by every retry.

Impact

Reward distribution can become increasingly expensive and may eventually require an upgrade or exceptional operational intervention to resume. The long inactivity prerequisite makes this best treated as a QA/liveness concern.

Recommendation

Add a permissionless chunked checkpoint that processes a bounded number of days per call, or replace daily zero-value snapshots with sparse expiry accounting that only visits days containing state changes.

Status

Resolved

Centralisation

The USD8 project values decentralisation alongside security and utility.

The protocol's functions are designed to operate independently, minimising reliance on the internal team and hardcoded values, while maintaining robust security and functionality.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the USD8 seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.